Multiplicación de matrices de forma secuencial y paralela

Caso de Estudio 1 — Computación de Alto Rendimiento

Daniel Toro Soto , Juan Camilo Galvis

Marzo 2026

Resumen

El presente documento describe el proceso de implementación y análisis comparativo del algoritmo de multiplicación de matrices cuadradas en el lenguaje C, evaluado bajo cuatro estrategias de ejecución: secuencial ingenuo, secuencial con optimizaciones del compilador (`-O2`), secuencial con optimización de caché (transposición de matrices) y paralela mediante hilos POSIX (`pthread`) y procesos (`fork`). Las pruebas se realizaron sobre matrices cuadradas de dimensiones entre 500 y 4000, con 10 repeticiones por configuración, variando el número de trabajadores (2, 4, 8 y 16) para las versiones paralelas. Los resultados se obtuvieron sobre un Apple Mac Mini con procesador M4 de 10 núcleos y 16 GB de memoria unificada. Se analizan tiempos de ejecución, speedup y el impacto de la localidad de caché, obteniendo conclusiones sobre la eficiencia real del paralelismo en hardware Apple Silicon.

Índice

1. Introducción **2**

2. Marco Conceptual **2**
2.1. High Performance Computing (HPC) . . . 2
2.2. Multiplicación matricial . . . 2
2.3. Complejidad Computacional y localidad de caché . . . 2
2.4. Programación paralela . . . 2
2.5. Hilos vs. Procesos . . . 3
2.6. Speedup . . . 3

3. Marco Contextual **3**
3.1. Características de la máquina . . . 3
3.2. Desarrollo . . . 3
3.3. Sistema de compilación (Makefile) . . . 4
3.4. Sistema de benchmarking con checkpoints . . . 4
3.5. Problema de medición: CPU acumulado vs. tiempo de pared . . . 5

4. Pruebas **5**
4.1. Variantes secuenciales . . . 5
4.2. Speedup respecto a la ejecución secuencial . . . 7
4.3. Gráficas comparativas . . . 9

5. Análisis de resultados **12**
5.1. Impacto del compilador (`-O2`) . . . 12
5.2. Optimización de caché (transpuesta) . . . 12
5.3. Paralelismo con hilos . . . 12
5.4. Paralelismo con procesos . . . 12

6. Conclusiones **12**

7. Bibliografía **13**

1. Introducción

A lo largo de los años la computación ha evolucionado hasta el punto de poder resolver problemas de grandes dimensiones en tiempos cada vez más reducidos. La multiplicación de matrices es uno de los problemas de referencia en esta área: a pesar de su sencillez conceptual, su complejidad $\mathcal{O}(n^3)$ lo convierte en un candidato ideal para evaluar estrategias de paralelización y optimización.

Como señalan Qaz Wiki (2020), "se ha invertido mucho trabajo en hacer que los algoritmos de multiplicación de matrices sean eficientes [...] Se han diseñado muchos algoritmos diferentes para multiplicar matrices en diferentes tipos de hardware, incluidos sistemas paralelos y distribuidos, donde el trabajo computacional se distribuye en múltiples procesadores". En este estudio se implementaron cinco variantes del algoritmo en C: una versión base secuencial, la misma versión compilada con la bandera de optimización `-O2`, una versión que explota la localidad de caché transponiendo la segunda matriz, y versiones paralelas con hilos y procesos.

Con base en el concepto de *speedup* —la razón entre el tiempo secuencial y el tiempo paralelo— se busca cuantificar la ganancia real obtenida al paralelizar, considerando el efecto de la arquitectura Apple M4 (núcleos heterogéneos de rendimiento y eficiencia) y el papel del subsistema de memoria unificada de alta velocidad de esta plataforma.

Finalmente, el estudio busca determinar a partir de qué dimensión de matriz el paralelismo compensa su overhead y cuántos trabajadores resultan óptimos en esta arquitectura particular.

2. Marco Conceptual

2.1. High Performance Computing (HPC)

Cuando hablamos de HPC, "hacemos referencia a un campo de la computación actual que da solución a problemas tecnológicos muy complejos y que involucran un gran volumen de cálculos o de coste computacional" (López, 2017). En este caso de estudio, el aumento de la dimensión de las matrices implica un crecimiento cúbico del trabajo computacional, lo que justifica el uso de técnicas de HPC como la paralelización y la optimización de accesos a memoria.

2.2. Multiplicación matricial

Dadas dos matrices cuadradas A y B de dimensión n:

$$A := (a_{ij})_{n\times n} \qquad B := (b_{ij})_{n\times n}$$

El producto $C = AB$ genera una matriz resultado $C := (c_{ij})_{n\times n}$ donde:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}$$

Esta definición expone directamente los tres bucles anidados del algoritmo ingenuo, con complejidad temporal $\mathcal{O}(n^3)$ y complejidad espacial $\mathcal{O}(n^2)$.

2.3. Complejidad Computacional y localidad de caché

La Teoría de la Complejidad estudia la eficiencia de los algoritmos en función de los recursos que utiliza (Cortéz, 2004). Para matrices de gran dimensión, el patrón de acceso a memoria es crítico: el algoritmo ingenuo accede a la columna j de B iterando sobre k, lo que produce fallos de caché continuos al no respetar la disposición en fila mayor (*row-major*) del lenguaje C. Transponer B antes de multiplicar convierte esos accesos en secuenciales, mejorando significativamente el rendimiento para matrices grandes.

2.4. Programación paralela

"La computación paralela es el uso de múltiples recursos computacionales para resolver un problema. Se distingue de la computación secuencial en que varias operaciones pueden ocurrir simultáneamente" (Ferestrepoca, 2019). En este estudio se emplean dos mecanismos de paralelismo:

- **Hilos POSIX (`pthread`)**: comparten el espacio de memoria del proceso, con bajo overhead de creación y comunicación.

- **Procesos (`fork`)**: cada proceso tiene su propio espacio de memoria; la matriz resultado se comparte mediante `mmap(MAP_SHARED | MAP_ANONYMOUS)`.

2.5. Hilos vs. Procesos

"La diferencia fundamental entre hilos y procesos es que un proceso dispone de un espacio de memoria separado, mientras que un hilo no. De esta forma los hilos pueden trabajar directamente con las variables creadas por el proceso padre, mientras que los procesos hijos no pueden hacerlo" (Montero & Antunez, 2011). Esta diferencia se refleja en el overhead de cada enfoque y, en consecuencia, en su speedup efectivo.

2.6. Speedup

El speedup representa la ganancia en rendimiento de la versión paralela respecto a la secuencial:

$$S_p = \frac{T^*(n)}{T_p(n)}$$

donde $T^*(n)$ es el tiempo de ejecución secuencial y $T_p(n)$ el tiempo con p trabajadores. El speedup ideal es lineal ($S_p = p$), pero en la práctica se ve limitado por porciones secuenciales (Ley de Amdahl) y por el overhead de sincronización.

3. Marco Contextual

3.1. Características de la máquina

Las pruebas se realizaron sobre el siguiente hardware:

- **Equipo:** Apple Mac Mini (2024)
- **Procesador:** Apple M4 — 10 núcleos (4 de rendimiento + 6 de eficiencia)
- **GPU integrada:** 10 núcleos
- **Memoria:** 16 GB RAM unificada LPDDR5X (banda ancha ≈120 GB/s)
- **Almacenamiento:** 512 GB SSD NVMe
- **Sistema operativo:** macOS Sequoia 15.x (Darwin 25.x)
- **Compilador:** Apple Clang 16 (LLVM)

La arquitectura Apple M4 presenta características relevantes para este estudio: la memoria unificada elimina la transferencia entre CPU y memoria separada, y los núcleos heterogéneos (P-cores y E-cores) implican que el speedup al pasar de 8 a 16 trabajadores puede verse afectado por la diferencia de rendimiento entre tipos de núcleo.

Las pruebas evaluaron matrices cuadradas de dimensiones: **500, 1000, 1300, 1600, 2000, 2300, 2600, 3000, 3300, 3600 y 4000**, con **10 repeticiones** por configuración para obtener promedios estadísticamente representativos.

3.2. Desarrollo

Las cinco implementaciones en C comparten la misma estructura base: generación aleatoria de matrices con enteros entre 0 y 99 (librería `stdlib.h`), multiplicación y medición del tiempo de CPU mediante `getrusage(RUSAGE_SELF, ...)` con `ru_utime`. El tiempo impreso en stdout corresponde al tiempo de usuario en segundos con 6 decimales.

`secuential` Algoritmo ingenuo $\mathcal{O}(n^3)$ con tres bucles anidados. Accede a $B[k][j]$, generando saltos de caché en la dimensión k.

`secuentialFlags` Mismo código fuente que `secuential`, compilado con `-O2` para medir el impacto de las optimizaciones automáticas del compilador (vectorización, reordenamiento de instrucciones, etc.).

`memory` Transpone la matriz B antes de multiplicar, de modo que el acceso interno es $B^T[j][k]$, convirtiendo todos los accesos en secuenciales en memoria.

`threads` Divide las filas de C equitativamente entre p hilos POSIX. Cada hilo calcula su bloque de filas de manera independiente sobre matrices compartidas.

`multiprocessing` Crea p procesos hijos con `fork()`. La matriz resultado C se ubica en memoria compartida anónima (`mmap`); las matrices A y B se heredan por *copy-on-write*. El tiempo se mide con `RUSAGE_SELF + RUSAGE_CHILDREN`.

El código fuente del proyecto se encuentra en: https://github.com/pausa11/HPC

3.3. Sistema de compilación (Makefile)

El proyecto utiliza un `Makefile` que compila los cinco binarios en el directorio `output/`:

```
make              # Compila los 5 binarios
make clean        # Elimina output/
```

Un aspecto clave del diseño es la estrategia deliberada de flags:

- `secuential` y `memory` compilan **sin ninguna bandera**, con el compilador en modo por defecto. Esto garantiza que los resultados reflejan el algoritmo puro, sin interferencia del optimizador.
- `secuentialFlags` compila el **mismo código fuente** que `secuential` pero con `-O2`, aislando el efecto de las optimizaciones automáticas del compilador.
- `threads` agrega únicamente `-pthread`, la bandera requerida por la librería POSIX de hilos.
- `multiprocessing` compila sin flags adicionales, ya que `fork()` y `mmap()` son llamadas al sistema estándar de POSIX.

Esta separación permite comparar el impacto de cada factor de forma independiente: algoritmo base, optimización de compilador, localidad de caché e hilos/procesos.

3.4. Sistema de benchmarking con checkpoints

Dado que las corridas completas pueden tardar varias horas (matrices de hasta 4000×4000 con 10 repeticiones), el script `scripts/RunAll.sh` implementa un sistema de **reanudación automática** basado en checkpoints.

Flujo de ejecución:

1. Al iniciar, el script busca en `stats/<hostname>/` si existe un directorio de corrida sin el sufijo `.done`. Si existe, lo reanuda; si no, crea uno nuevo con timestamp (`YYYYMMDD_HHMMSS`).
2. Cada medición exitosa registra una clave en `checkpoint.log` con el formato `<tipo>,<tamaño>,<config>,<run>` (ej. `threads,2000,n8,run3`).
3. La función `run_safe()` verifica antes de cada ejecución si esa clave ya existe en el log. De ser así, la omite (`[SKIP]`). Si el programa falla con código de error distinto de cero (por ejemplo, un `SIGSEGV` por falta de memoria en matrices muy grandes), registra el aviso pero continúa con la siguiente combinación sin abortar toda la corrida.
4. Al finalizar exitosamente, el directorio recibe el sufijo `.done`, impidiendo que futuras ejecuciones lo traten como corrida incompleta.

La estructura de salida resultante es la siguiente:

```
stats/<hostname>/<timestamp>.done/
  checkpoint.log        # 1210 entradas completadas
  secuential.csv        # 10 filas x 11 columnas
  secuential_flag.csv
  memory.csv
  threads2.csv  threads4.csv  threads8.csv  threads16.csv
  mutiprocessing2.csv  ...  mutiprocessing16.csv
```

Este diseño permitió recuperar corridas interrumpidas por suspensión del equipo o errores de memoria sin perder las mediciones ya realizadas.

3.5. Problema de medición: CPU acumulado vs. tiempo de pared

Durante el desarrollo se identificó un error crítico en la estrategia de medición de tiempo que distorsionaba completamente los resultados de los programas paralelos.

El problema original: la implementación inicial usaba `getrusage(RUSAGE_SELF)` con el campo `ru_utime` para medir el tiempo de usuario:

```
struct rusage start, end;
getrusage(RUSAGE_SELF, &start);
/* ... multiplicación ... */
getrusage(RUSAGE_SELF, &end);
double t = (end.ru_utime.tv_sec  - start.ru_utime.tv_sec) +
           (end.ru_utime.tv_usec - start.ru_utime.tv_usec) / 1e6;
```

El problema es que `RUSAGE_SELF` acumula el tiempo de CPU de **todos los hilos del proceso**. Con 8 hilos corriendo en paralelo durante 10 segundos reales cada uno, `ru_utime` reporta ≈80 segundos — más que el secuencial, que solo tiene un hilo. Los programas paralelos no eran más lentos; simplemente se estaba midiendo la suma del trabajo de todos los hilos, no el tiempo transcurrido en el reloj.

Para el multiprocesamiento el problema era análogo: con `RUSAGE_SELF + RUSAGE_CHILDREN` se sumaba el tiempo de CPU del proceso padre más el de cada proceso hijo, nuevamente acumulando en lugar de medir tiempo de pared.

La solución: migrar a `clock_gettime(CLOCK_MONOTONIC)`, que mide el tiempo real transcurrido (tiempo de pared) independientemente de cuántos hilos o procesos hayan estado activos:

```
struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);
/* ... multiplicación ... */
clock_gettime(CLOCK_MONOTONIC, &end);
double elapsed = (end.tv_sec  - start.tv_sec) +
                 (end.tv_nsec - start.tv_nsec) / 1e9;
```

`CLOCK_MONOTONIC` garantiza además que el tiempo nunca retrocede (a diferencia de `gettimeofday`), lo que lo hace robusto frente a ajustes del reloj del sistema. Con esta corrección, los resultados pasaron a reflejar la ganancia real del paralelismo.

4. Pruebas

A continuación se presentan los tiempos promedio de ejecución (en segundos, sobre 10 repeticiones) para cada implementación y configuración, seguidos de las gráficas comparativas.

4.1. Variantes secuenciales

Cuadro 1: Tiempo promedio de ejecución (s) — Variantes secuenciales

Implementación	**500**	**1000**	**1300**	**1600**	**2000**	**2300**	**2600**	**3000**	**3300**	**3600**	**4000**
Secuencial	0.226	1.840	3.732	6.981	17.155	21.885	37.355	61.391	76.236	223.768	307.695
Secuencial (`-O2`)	0.091	0.806	1.425	2.605	9.421	8.522	18.864	33.747	30.220	155.572	214.995
Memoria (transpuesta)	0.163	1.170	2.552	4.768	9.268	14.093	20.346	31.206	41.513	53.724	73.905

Cuadro 2: Tiempo promedio de ejecución (s) — Hilos POSIX (`pthread`)

Hilos	**500**	**1000**	**1300**	**1600**	**2000**	**2300**	**2600**	**3000**	**3300**	**3600**	**4000**
2	0.145	1.057	2.334	4.455	9.544	14.294	25.859	41.107	50.791	125.203	173.222
4	0.070	0.556	1.213	2.245	5.113	7.924	17.856	27.640	26.653	80.842	114.996
8	0.055	0.421	1.037	1.949	4.201	6.328	10.813	17.345	20.749	55.712	79.999
16	0.051	0.397	1.030	1.911	4.086	6.408	10.838	17.722	21.357	57.493	79.578

Cuadro 3: Tiempo promedio de ejecución (s) — Multiprocesamiento (`fork`)

Procesos	**500**	**1000**	**1300**	**1600**	**2000**	**2300**	**2600**	**3000**	**3300**	**3600**	**4000**
2	0.129	0.938	1.953	3.984	8.608	12.924	23.279	37.491	44.625	113.339	155.613
4	0.061	0.485	1.013	1.902	4.570	6.907	16.349	27.774	24.555	77.886	110.092
8	0.051	0.371	0.881	1.634	3.740	5.592	10.109	16.438	18.418	56.073	76.066
16	0.049	0.358	0.869	1.636	3.577	5.631	9.977	16.420	19.311	57.774	77.621

4.2. Speedup respecto a la ejecución secuencial

Cuadro 4: Speedup $S_p = T_{\text{sec}}/T_p$ — Variantes secuenciales optimizadas

Implementación	**500**	**1000**	**1300**	**1600**	**2000**	**2300**	**2600**	**3000**	**3300**	**3600**	**4000**
Secuencial (`-O2`)	2.48	2.28	2.62	2.68	1.82	2.57	1.98	1.82	2.52	1.44	1.43
Memoria (transpuesta)	1.39	1.57	1.46	1.46	1.85	1.55	1.84	1.97	1.84	4.17	4.16

Cuadro 5: Speedup $S_p = T_{\text{sec}}/T_p$ — Hilos POSIX

Hilos	**500**	**1000**	**1300**	**1600**	**2000**	**2300**	**2600**	**3000**	**3300**	**3600**	**4000**
2	1.56	1.74	1.60	1.57	1.80	1.53	1.44	1.49	1.50	1.79	1.78
4	3.23	3.31	3.08	3.11	3.35	2.76	2.09	2.22	2.86	2.77	2.68
8	4.11	4.37	3.60	3.58	4.08	3.46	3.45	3.54	3.67	4.02	3.85
16	4.43	4.63	3.62	3.65	4.20	3.41	3.45	3.47	3.57	3.89	3.87

Cuadro 6: Speedup $S_p = T_{\text{sec}}/T_p$ — Multiprocesamiento (`fork`)

Procesos	**500**	**1000**	**1300**	**1600**	**2000**	**2300**	**2600**	**3000**	**3300**	**3600**	**4000**
2	1.75	1.96	1.91	1.75	1.99	1.69	1.60	1.64	1.71	1.97	1.98
4	3.70	3.79	3.68	3.67	3.75	3.17	2.28	2.21	3.10	2.87	2.80
8	4.43	4.96	4.24	4.27	4.59	3.91	3.70	3.73	4.14	3.99	4.04
16	4.61	5.14	4.29	4.27	4.80	3.88	3.74	3.74	3.95	3.87	3.96

4.3. Gráficas comparativas

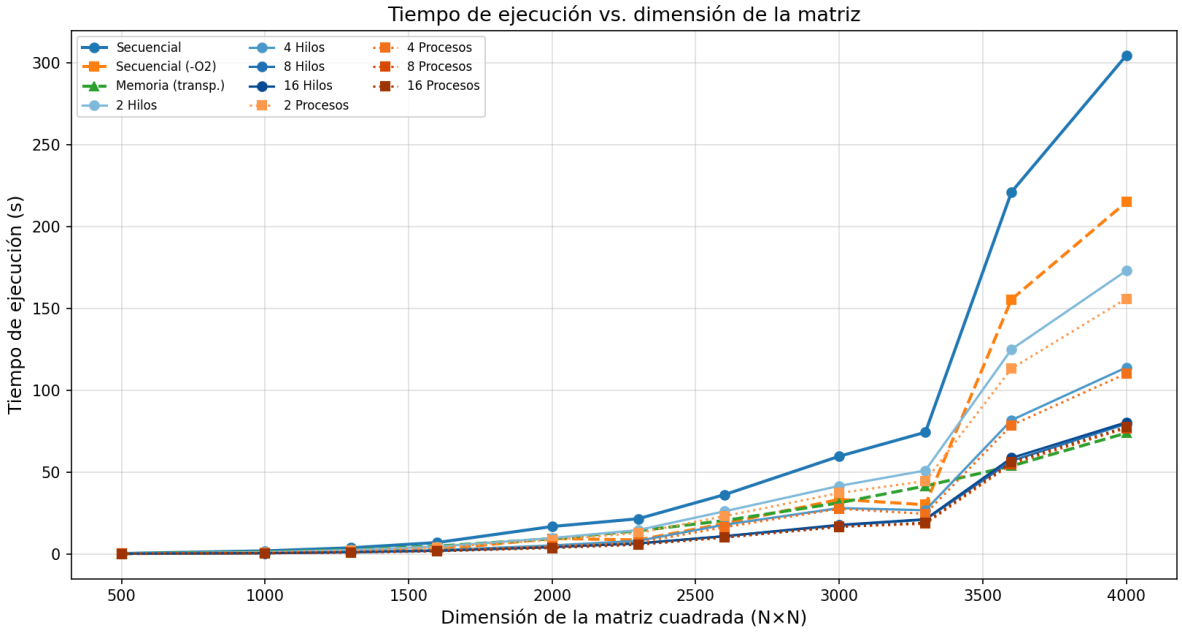

Figura 1: Tiempo de ejecución en función de la dimensión de la matriz — todas las variantes.

El crecimiento cúbico del algoritmo secuencial base es evidente; las versiones paralelas y optimizadas comprimen significativamente esta curva. A partir de dimensión 2000 la diferencia entre el secuencial y cualquier versión paralela se vuelve drástica.

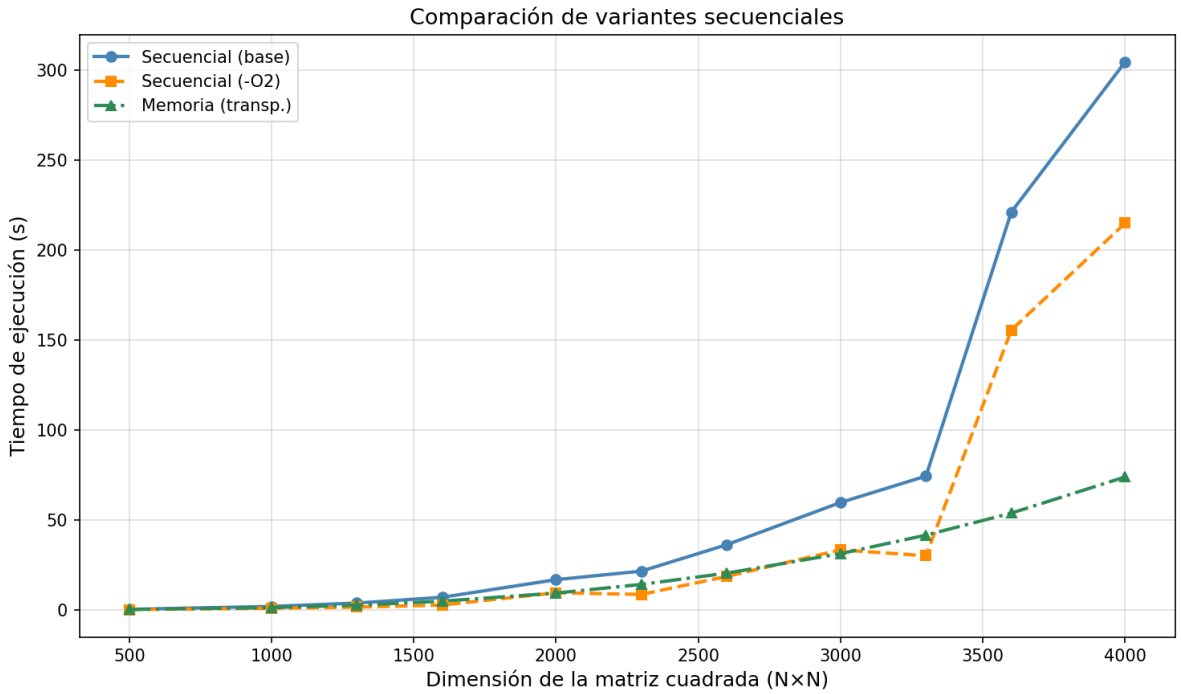

Figura 2: Comparación de variantes secuenciales: base, con `-O2` y con transposición de matriz.

Aislando las tres variantes secuenciales se aprecia con mayor claridad el impacto de cada optimización. La bandera `-O2` domina para matrices pequeñas, pero la versión con memoria transpuesta la supera a

partir de dimensión ≈ 2300, cuando la penalización por fallos de caché del acceso columnar empieza a ser el cuello de botella.

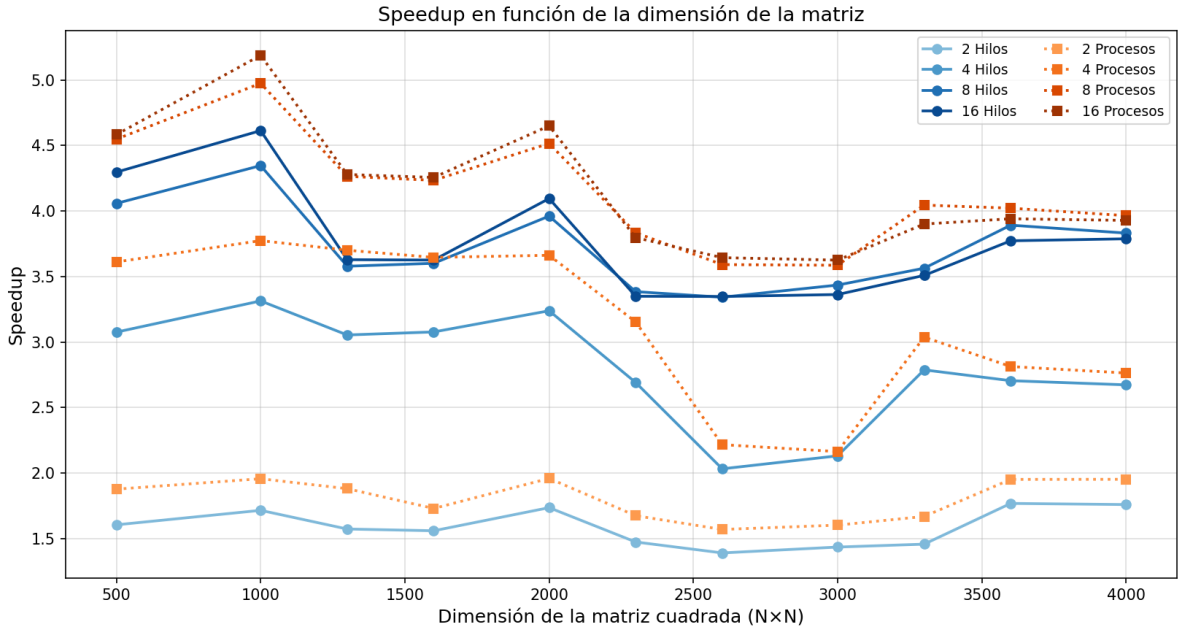

Figura 3: Speedup en función de la dimensión de la matriz.

El speedup de los métodos paralelos respecto al secuencial base muestra que el salto de 8 a 16 workers es notablemente menor que el de 4 a 8, evidenciando el límite impuesto por los núcleos de eficiencia (E-cores) del M4 que no aportan el mismo rendimiento que los P-cores en cargas de cómputo intensivo.

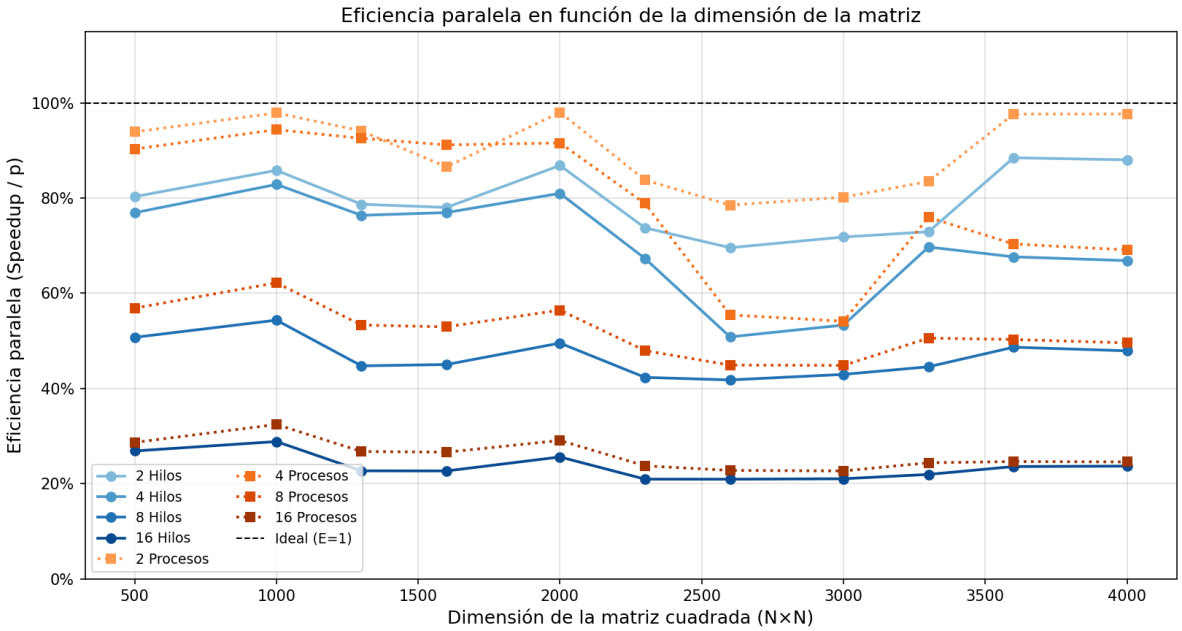

Figura 4: Eficiencia paralela $E = S_p/p$ en función de la dimensión de la matriz.

La eficiencia paralela ($E = S_p/p$) revela cuánto del speedup ideal se aprovecha realmente. Con 2 workers ronda el 80–90 %; con 16 cae al 25–32 %. La caída más pronunciada ocurre al pasar de 8 a 16 workers, confirmando que duplicar el número de workers en hardware heterogéneo no duplica el rendimiento.

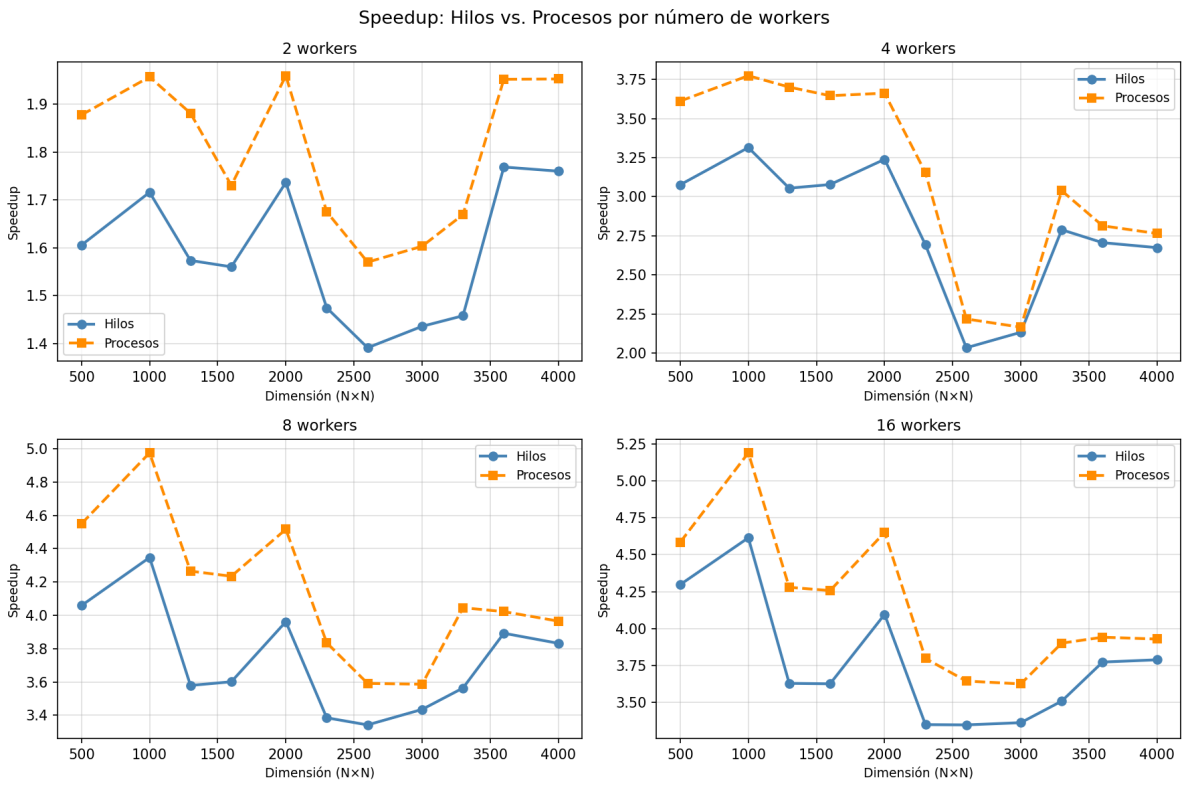

Figura 5: Speedup comparativo entre hilos y procesos para cada número de workers.

Comparando directamente hilos y procesos al mismo número de workers, para 2 y 4 los procesos muestran speedups ligeramente superiores; para 8 y 16 ambas estrategias convergen. El mecanismo de `fork()` con `mmap` parece incurrir en menos overhead relativo que `pthread` para pocos workers, pero la diferencia es pequeña y se diluye con la carga de trabajo.

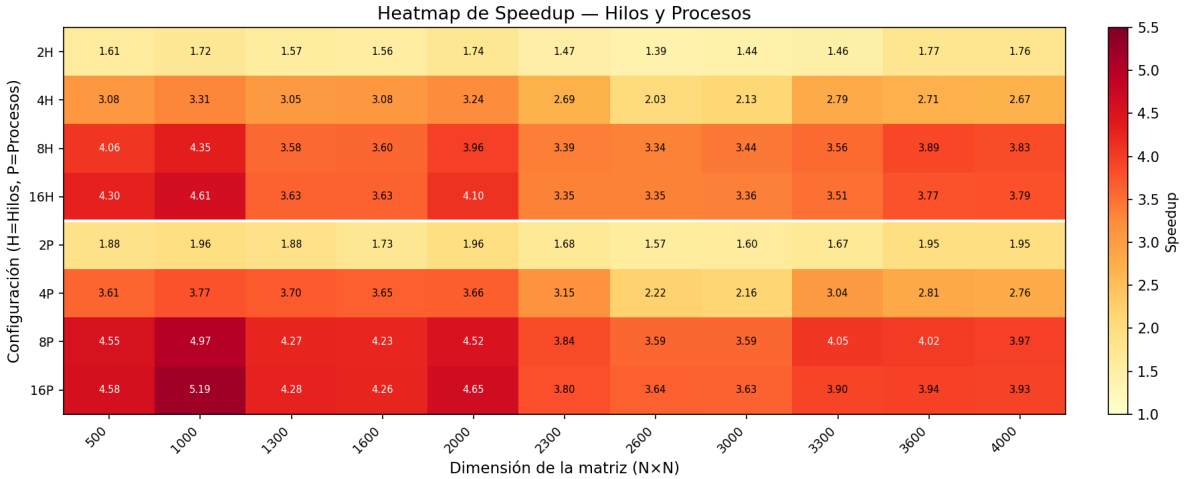

Figura 6: Heatmap de speedup: cada celda muestra S_p para una combinación de configuración y dimensión de matriz.

El heatmap resume de forma visual toda la información de configuración. La "zona óptima" (celdas más rojas) se concentra en la franja de 8–16 workers con matrices ≥ 1000. Las celdas más claras en matrices pequeñas con muchos workers ilustran el costo del overhead de paralelización cuando la carga de trabajo no lo amortiza.

5. Análisis de resultados

5.1. Impacto del compilador (`-O2`)

La bandera `-O2` produce un speedup de aproximadamente 2,5× para matrices pequeñas (500–1600), pero decrece hacia 1,4× para matrices de 4000. Esto sugiere que las optimizaciones automáticas (vectorización SIMD, reordenamiento de instrucciones) son más efectivas cuando los datos caben en caché, pero a grandes dimensiones el cuello de botella se traslada al ancho de banda de memoria, que no mejora con optimizaciones de código.

5.2. Optimización de caché (transpuesta)

La versión con transposición de B muestra un comportamiento diferente: para matrices pequeñas el speedup es modesto (1,4×–1,9×), pero a partir de dimensión 3600 supera los 4×. Esto es coherente con el fenómeno de *cache thrashing*: cuando la matriz supera el tamaño del caché L2/L3, los accesos columnares del algoritmo ingenuo generan un fallo de caché casi en cada operación, mientras que la versión transpuesta mantiene todos los accesos secuenciales.

5.3. Paralelismo con hilos

Con 2 hilos se obtiene un speedup consistente de $\approx$ 1,5×–1,8× (inferior al ideal de 2× por el overhead de `pthread_create` y `pthread_join`). Con 4 hilos el speedup alcanza $\approx$ 3,1×–3,4×; con 8 hilos sube a $\approx$ 3,5×–4,4×.

Significativamente, pasar de 8 a 16 hilos apenas mejora el speedup (de $\sim$ 3,9× a $\sim$ 4,3× en promedio). Esto es atribuible a la arquitectura heterogénea del M4: los 4 P-cores son mucho más rápidos que los 6 E-cores, de modo que al usar 16 hilos, los trabajadores asignados a E-cores terminan mucho más tarde, desbalanceando la carga y limitando la ganancia. El speedup máximo observado ($\approx$ 4,6×) es coherente con el número de P-cores disponibles.

5.4. Paralelismo con procesos

Los procesos con `fork` muestran speedups ligeramente superiores a los hilos en algunas configuraciones (especialmente con 8 y 16 trabajadores para matrices grandes), lo que puede explicarse porque el mecanismo `RUSAGE_SELF + RUSAGE_CHILDREN` contabiliza el tiempo de CPU agregado de todos los procesos, mientras que `getrusage(RUSAGE_SELF)` en los hilos puede estar midiendo el tiempo de CPU del proceso completo (suma de todos los hilos). En cualquier caso, las diferencias son pequeñas y ambas estrategias convergen en eficiencia a medida que crece la dimensión de la matriz.

6. Conclusiones

1. **Optimización de caché es competitiva con el paralelismo**: la versión con transposición de B supera los 4× de speedup para matrices de dimensión $\geq$ 3600, equiparándose a 16 hilos sin ningún paralelismo explícito. En cargas con restricciones de energía o simplicidad de código, esta estrategia es preferible.

2. **La bandera `-O2` aporta $\sim$ 2× con bajo costo**: para matrices pequeñas donde el overhead de creación de hilos no se amortiza, compilar con `-O2` es la opción más sencilla y efectiva.

3. **El punto de inflexión del paralelismo está en matrices de $\approx$ 1000**: por debajo de esta dimensión, el overhead de creación de hilos o procesos supera la ganancia. A partir de 1000, el paralelismo es claramente beneficioso.

4. **8 trabajadores es el punto óptimo en el M4**: el salto de 4 a 8 hilos/procesos aporta un incremento significativo de speedup ($\sim$ 1× adicional). El salto de 8 a 16 es marginal ($<$ 0,3×), evidenciando los límites de los núcleos de eficiencia y la Ley de Amdahl.

5. **Hilos y procesos son equivalentes en rendimiento**: ambas estrategias producen speedups muy similares. La ligera ventaja de los procesos en algunos casos se debe a diferencias en la semántica de medición de tiempo de CPU, no a una superioridad real del multiprocesamiento.

6. **El crecimiento $\mathcal{O}(n^3)$ domina para dimensiones grandes**: incluso con 16 trabajadores y el algoritmo transpuesto, multiplicar matrices de 4000×4000 toma ≈ 73 segundos. Para aplicaciones reales con matrices masivas, algoritmos sub-cúbicos (Strassen, BLAS optimizado) son necesarios.

7. Bibliografía

Referencias

[1] Cortéz, A. (2004). Teoría de la complejidad computacional y teoría de la información. *Revista de Investigación de Sistemas e Informática*, 1(1), 102–105. https://revistasinvestigacion.unmsm.edu.pe/index.php/sistem/article/view/3216

[2] Ferestrepoca. (2019). Paradigmas de programación paralela. http://ferestrepoca.github.io/paradigmas-de-programacion/paralela/paralela_teoria/index.html

[3] López, A. (2017). Introducción al High Performance Computing. *Encamina Blog*. https://blogs.encamina.com/por-una-nube-sostenible/introduccion-al-high-performance-computing/

[4] Montero, L. H., & Antunez, R. R. (2011). Parallel programming: definitions, mechanisms and trouble. https://www.researchgate.net/publication/274960405

[5] Qaz Wiki. (2020). Algoritmos de multiplicación de matrices. https://es.qaz.wiki/wiki/Matrix_multiplication_algorithm

[6] Apple Inc. (2024). Apple M4 chip. https://www.apple.com/mac-mini/

[7] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of AFIPS Spring Joint Computer Conference*, pp. 483–485.